PRÁCTICA 1: DESARROLLO DE SERVIDOR CON SOCKETS EN JAVA Y CREACIÓN DE GRÁFICAS

Autor: **Juan Manuel Orduña Huertas**

1- OBJETIVO Y DESCRIPCIÓN GENERAL.

El objetivo de esta práctica es por un lado ver un ejemplo de desarrollo de servidor multicliente en Java usando sockets TCP, como ayuda para el desarrollo del proyecto de la asignatura.

La segunda parte de la práctica será un ejemplo práctico sobre cómo obtener las gráficas de prestaciones que hay que analizar en la última sesión del proyecto a partir de los resultados en fichero de texto que proporciona un programa de simulación, que podría ser el proyecto desarrollado por el alumno.

2- DESARROLLO DE SERVIDOR TCP MULTICLIENTE EN JAVA

2.1- Mini-servidor local de órdenes.

Como ejemplo inicial de comunicación servidor-cliente, vamos a realizar un servidor local de órdenes, ubicado en la propia máquina. Para ello, usaremos la dirección de loopback 127.0.0.1. La aplicación a realizar se llamará *server_ordenes*, y depende de una clase de comunicaciones proporcionada por Java: ServerSocket. Esta clase realiza la mayor parte del trabajo de crear un servidor. El servidor implementado escucha por el puerto 8001, y acepta las órdenes HELP, QUIT, DATE y NAME.

La orden HELP primera hará que el servidor devuelva al cliente un mensaje con la sintaxis o nombre de las órdenes que puede aceptar. La segunda servirá para finalizar tanto el servidor como el cliente, y las otras dos harán que el servidor ejecute el comando homónimo del shell y devuelva el resultado al cliente.

El cliente puede desarrollarse también como un programa cliente (ejercicio opcional), o bien se puede utilizar la aplicación telnet en un terminal distinto a la que esté ejecutando el servidor, mediante el comando *telnet 127.0.0.1 8001*.

En el directorio de trabajo elegido para desarrollar la práctica, hay que copiar el siguiente código y grabarlo como fichero de texto con el nombre “server_ordenes.java”:

```
import java.util.*;
import java.io.*;
import java.net.*;

public class server_ordenes {
```

```
final static int SERVER_PORT = 8001; // puerto de escucha de nuestro servidor

public static void main (String args[]) {
Server server;
String clientRequest;
BufferedReader reader;
PrintWriter writer;

// creamos el servidor y los canales de entrada y salida

server = new Server (SERVER_PORT);
reader = new BufferedReader (new InputStreamReader (server.in));
writer = new PrintWriter (new OutputStreamWriter (server.out), true);



// En cuanto se establece una conexión por parte del cliente, enviamos un saludo
writer.println ("Bienvenido al Servidor: " + new Date() + "/n");

while (true) {
try {
// leemos del canal de entrada la petición del cliente
clientRequest = reader.readLine();

// Sacamos por pantalla la peticion del cliente
System.out.println ("Recibido :" + clientRequest);

// El protocolo de nuestro servidor solo acepta ordenes : HELP, QUIT,NAME,DATE
if (clientRequest.startsWith ("HELP")) {
writer.println ("Órdenes: HELP QUIT NAME DATE");
} else {
if (clientRequest.startsWith ("QUIT")) {
System.exit(0);
}
else {
if (clientRequest.startsWith("NAME"))
{InetAddress host;
host=InetAddress.getLocalHost();
writer.println("Nombre del host :"+ host.getHostName());}
else
if (clientRequest.startsWith("DATE"))
{writer.println("Fecha del sistema :"+ new Date());}

else {
writer.println ("ERROR: Comando :'" + clientRequest +"' no reconocido, use HELP");
}
}
}
} catch (IOException e) {
System.out.println ("Excepción en el servidor " + e);
System.exit(0);

}
}
}
}
//---------------------------------------------------------------------

// Esta clase es la que implementa el socket del servidor (ServerSocket)
class Server {

private ServerSocket server;
private Socket socket;

public InputStream in;
public OutputStream out;

public Server (int port) {

try {
server = new ServerSocket (port);
System.out.println ("Servidor Java Activo! \n");
```

```
System.out.println(""+server+"\n");
// Espera suspendido hasta que un cliente establece una conexión
socket = server.accept();


in = socket.getInputStream();
out = socket.getOutputStream();

} catch (IOException e) {
System.out.println ("Excepción en el constructor server: " + e);
}
}
}
```

Para compilar el programa, en una terminal hay que cambiar al directorio de trabajo, y una vez allí ejecutar el compilador:

```
# cd <ruta al directorio de trabajo>

# javac server_ordenes.java
```

Para ejecutar el programa servidor ejecutaremos el comando

```
# java server_ordenes
```

Una vez esté en marcha el servidor, abriremos otra terminal y ejecutaremos el comando

```
# telnet 127.0.0.1 8001
```

Con lo que podremos ejecutar las órdenes que queramos desde el cliente.

Una vez probado que el servidor funciona, se pasará a capturar y analizar con Wireshark los segmentos que se intercambian entre el cliente y el servidor (véase la Sección 5 para información sobre el uso de Wireshark). Por comodidad, se debería configurar Wireshark para usar el interfaz loopback, cuya dirección IP es 127.0.0.1. Se deberá ejecutar el servidor y el cliente, tecleando cada vez uno de los comandos que acepta el servidor, hasta teclear en último lugar el comando QUIT.

Una vez ejecutado el programa de esta forma, los alumnos deberán analizar la información suministrada por el Wireshark y responder a las siguientes preguntas:

1- Detectar los segmentos que conforman el saludo a tres vías para la conexión del socket, y enumerar los flags que tiene activos dichos segmentos y el número de secuencia que tiene cada uno de estos segmentos.

2- Detectar el tamaño de la ventana que anuncia el servidor en cada una de las respuestas que da al cliente.

3- Detectar el tamaño de los datos enviados por el cliente en cada uno de los segmentos que envía.

4- Detectar los segmentos que conforman la despedida a tres vías, y comprobar si se ha cerrado correctamente la conexión.

A continuación, se deberá repetir el ejercicicio anterior, pero antes de teclear qel comando QUIT abriremos una nueva terminal y ejecutaremos otro cliente contra el mimsmo servidor. Observar las tramas enviadas y recibidas, así como los mensajes que se obtienen en la terminal.

5- ¿Se realiza la apertura de un nuevo socket en el servidor mediante el saludo a 3 vías?

6- ¿Se realiza la transmisión de algún segmento por el socket? ¿Cuántos, de qué socket a qué socket qué información llevan?

2.2- Servidor multi-cliente.

Como conclusión de lo que hemos visto en el apartado anterior, podemos decir que es fácil implementar la comunicación de un servidor con un cliente. El problema comienza a complicarse cuando se desea la comunicación simultánea de un servidor con dos o más clientes. En esa parte de la práctica vamos a desarrollar y probar un servidor multicliente, y analizaremos las comunicaciones que realizan entre ellos. Para ello, crearemos un nuevo directorio de trabajo.

Para que un servidor atienda a la vez a varios clientes utilizaremos threads o hilos. Para cada cliente que se conecte al servidor crearemos un nuevo hilo en el mismo que se encargará de gestionar el cliente. Este receptor lo representamos con la clase Java **ServidorHilo.java**. Es decir, en el nuevo directorio de trabajo crearemos un fichero con el nombre “ServidorHilo.java”, cuyo código será el siguiente:

```java
import java.io.*;
import java.net.*;
import java.util.logging.*;
public class ServidorHilo extends Thread {
   private Socket socket;
   private DataOutputStream dos;
   private DataInputStream dis;
   private int idSessio;
   public ServidorHilo(Socket socket, int id) {
      this.socket = socket;
      this.idSessio = id;
      try {
         dos = new DataOutputStream(socket.getOutputStream());
         dis = new DataInputStream(socket.getInputStream());
      } catch (IOException ex) {
         Logger.getLogger(ServidorHilo.class.getName()).log(Level.SEVERE, null, ex);
      }
   }
   public void desconnectar() {
      try {
         socket.close();
      } catch (IOException ex) {
         Logger.getLogger(ServidorHilo.class.getName()).log(Level.SEVERE, null, ex);
      }
   }
   @Override
   public void run() {
      String accion = "";
      try {
         accion = dis.readUTF();
         if(accion.equals("hola")){
            System.out.println("El cliente con idSesion "+this.idSessio+" saluda");
            dos.writeUTF("adios");
         }
      } catch (IOException ex) {
         Logger.getLogger(ServidorHilo.class.getName()).log(Level.SEVERE, null, ex);
      }
      desconnectar();
   }
}
```

En este código se puede observar que en el constructor creamos dos buffers (uno de entrada y otro de salida) para gestionar los envíos y recibos del cliente, y dentro del método RUN gestionamos qué hacer cuando recibamos un mensaje del cliente.

A continuación crearemos el fichero “Cliente.java” e introduciremos el siguiente código, que crea 5 clientes que simplemente envían un saludo y reciben la respuesta del servidor a cada cliente, imprimiéndola por pantalla:

```java
import java.io.*;
import java.net.Socket;
import java.util.*;
import java.util.logging.*;
class Persona extends Thread {
   protected Socket sk;
   protected DataOutputStream dos;
   protected DataInputStream dis;
   private int id;
   public Persona(int id) {
      this.id = id;
   }
   @Override
   public void run() {
      try {
         sk = new Socket("127.0.0.1", 10578);
         dos = new DataOutputStream(sk.getOutputStream());
         dis = new DataInputStream(sk.getInputStream());
         System.out.println(id + " envía saludo");
         dos.writeUTF("hola");
         String respuesta="";
         respuesta = dis.readUTF();
         System.out.println(id + " Servidor devuelve saludo: " + respuesta);
         dis.close();
         dos.close();
         sk.close();
      } catch (IOException ex) {
         Logger.getLogger(Persona.class.getName()).log(Level.SEVERE, null, ex);
      }
   }
}
public class Cliente {
   public static void main(String[] args) {
      ArrayList<Thread> clients = new ArrayList<Thread>();
      for (int i = 0; i < 5; i++) {
         clients.add(new Persona(i));
      }
      for (Thread thread : clients) {
         thread.start();
      }
   }
}
```

Hay que destacar que por simplicidad es el propio código del cliente el que lleva la IP del servidor. Si queremos ejecutar el cliente en la misma máquina que el servidor la IP será la de loopback (127.0.0.1), pero si queremos ejecutar el cliente contra un servidor remoto, deberemos averiguar al IP del servidor (usando en una terminal de este el comando **`# ip address`**) y poner en el código del cliente dicha dirección IP.

Finalmente, crearemos la clase **Servidor.java**, que se encarga de asignar un nuevo Thread a cada nueva conexión. Para ello, crearemos el fichero “Servidor.java” y escribiremos el siguiente código, que crea el hilo e imprime por pantalla los datos de cada nueva conexión:

```java
import java.io.*;
import java.net.*;
import java.util.logging.*;
public class Servidor {
   public static void main(String args[]) throws IOException {
      ServerSocket ss;
      System.out.print("Inicializando servidor... ");
      try {
         ss = new ServerSocket(10578);
         System.out.println("\t[OK]");
         int idSession = 0;
         while (true) {
            Socket socket;
            socket = ss.accept();
            System.out.println("Nueva conexión entrante: "+socket);
            ((ServidorHilo) new ServidorHilo(socket, idSession)).start();
            idSession++;
         }
      } catch (IOException ex) {
         Logger.getLogger(Servidor.class.getName()).log(Level.SEVERE, null, ex);
      }
   }
}
```

Una vez creados estos ficheros, los compilaremos con el comando

```
# javac *.java
```

A continuación ejecutaremos el programa servidor mediente el comando

```
# java Servidor
```

Una vez esté en marcha el servidor, podemos abrir otra terminal (en el mismo computador o en otro computador remoto, según hayamos escrito en código del cliente) y ejecutaremos el comando

```
# java Cliente
```

Con esto se deberían ejecutar los clientes determinados en el Cliente, enviando cada uno un saludo y recibiendo una contestación del servidor.

Una vez comprobado que todo funciona, abrir de nuevo el programa Wireshark y ejecutar el Cliente con 5 clientes (ojo, si ejecutamos el programa cliente desde el mismo computador tendremos que seleccionar en Wireshark el interfaz de loopback, pero si usamos un computador remoto tendremos que usar la interfaz de red, usualmente denominada (en distribuciones linux) “enp2s0” o algo similar. Preguntar al profesor en caso de duda). Responder a las siguientes cuestiones:

7- Realiza una captura de pantalla de Wireshark y según esa pantalla contesta si el saludo a tres vías a cada uno de los clientes se ha hecho de forma secuencial o si por el contrario se han ido mezclando en el tiempo segmentos de los distintos clientes.

8- Identifica los sockets implicados en las comunicaciones con los 5 clientes.

9- Identifica los números de secuencia utilizados en las comunicaciones de cada socket. ¿Coinciden, o en cada socket se han utilizado distintos ISNs?

10- Modifica el cliente para que aumentar el número de conexiones que se realizan contra el servidor y ejecuta el cliente en un computador remoto. Aumenta el número de conexiones hasta que se produzcan errores en las conexiones. Indica cuántas conexiones ha soportado el servidor.

3- GENERACIÓN DE GRÁFICAS DE PRESTACIONES A PARTIR DE LA SALIDA DEL PROGRAMA DEL PROYECTO

3.1- Cálculo de latencia, tráfico inyectado y tráfico aceptado.

El proyecto de la asignatura consiste en un servidor multicliente que simula el tráfico generado por las comunicaciones de un sistema de Realidad Aumentada Colaborativa, donde tenemos 1 servidor, N clientes, V vecinos de cada cliente, y la generación del tráfico y las respuestas correspondientes se generan I iteraciones, donde N, V e I son parámetros de entrada introducidos por teclado, y conocidos tanto por el servidor como por los clientes.

3.1.1 Cálculo de latencia

Al final de la simulación CADA cliente debe tener calculada la media del tiempo de respuesta de sus vecinos después de la I iteraciones. Antes de desconectarse debe enviar es media al servidor, y este calculará la media del tiempo de respuesta de todos los clientes para esa simulación, y la imprimirá por pantalla Y EN UN FICHERO DE RESULTADOS. Esa media será el tiempo de respuesta o latencia que usaremos para construir la gráfica de latencia. Supongamos que el valor de esa media es de 11,234875 segundos.

3.1.2 Cálculo de tráfico inyectado

No obstante, para construir la gráfica de latencia necesitamos tener en el eje de abcisas el tráfico inyectado en la red, en términos de bytes (o bits) por unidad de tiempo. Por eso el proyecto debe calcular en cada simulación (además de la media de tiempo de respuesta explicado en el párrafo anterior) cuántos bytes (o bits) han enviado todos los clientes en esa simulación, y cuánto tiempo ha tardado EN EJECUTARSE la simulación desde el momento en que el servidor envió la señal de comienzo de simulación a todos los clientes, hasta el instante que terminó de ejecutarse el último cliente.

Supongamos que la simulación se ha ejecutado con N=1000 clientes, V = 100 vecinos en cada grupo, e I = 10 iteraciones, y supongamos que la ejecución de la simulación ha requerido 234.560 milisegundos. El cálculo del tráfico inyectado que corresponderá a la latencia obtenida será como sigue: En cada iteración cada cliente habrá generado 99 mensajes con sus nuevas coordenadas, y habrá recibido 99 mensajes con el reconocimiento. El tamaño de los mensajes es conocido. Supongamos que los mensajes con nuevas coordenadas son de 100 bytes, y los de reconocimiento tienen 30 bytes de longitud. Por tanto, cada cliente ha generado en cada iteración 99 x 100 bytes + 99 x 30 bytes = 9.900 + 2.970 = 12.870 bytes. Como hay 1000 clientes, en cada iteración se han genera-

do 12.870.000 bytes, y en toda la simulación (10 iteraciones) se habrán generado 128.700.000 bytes. Como la simulación ha tardado 234.560 milisegundos (234,560 segundos), el tráfico inyectado ha sido de 128.700.000 bytes / 234,560 seg. = 548.686,903 bytes/seg.

Así pues, esta simulación habrá generado un punto de la gráfica de latencia que tendrá el valor en el eje de abcisas (tráfico inyectado) de 548.686,903 bytes/segundo (o 548,686903 bytes/cliente/segundo, ya que hay 1000 clientes), y el valor en el eje de ordenadas de 11,234875 segundos.

3.1.2 Cálculo de tráfico aceptado (throughput)

El tráfico aceptado es el tráfico que ha llegado a su destino durante la simulación. También se le conoce como *throughput.* Esta medida de prestaciones la tiene que proporcionar el programa que desarrolle el equipo, y básicamente consiste en que cada cliente lleve la cuenta de bytes que le llegan, ya sea de nuevas coordenadas de sus vecinos o de reconocimientos de sus vecinos a coordenadas enviadas por él. Si el equipo desea que su proyecto proporcione gráfica de throughput o tráfico aceptado, además de la media de latencia CADA CLIENTE deberá enviar al servidor al final de la simulación no sólo la media de latencia, sino el total de bytes que ha recibido él en la simulación. El servidor deberá **sumar** todos los bytes recibidos por todos los clientes.

Supongamos que la simulación anterior, ejecutada con N=1000 clientes, V = 100 vecinos en cada grupo, e I = 10 iteraciones, resulta en que entre todos los clientes le envían al servidor una suma de 128.700.000 bytes, de los cuales 127.960.000 bytes han llegado a su destino. Si esa simulación tardó en ejecutarse 234,560 segundos, resultará por tanto en un punto de la gráfica de throughput que tendrá el valor en el eje de abcisas (tráfico inyectado) de 128.700.000 bytes / 234,560 segundos = 548.686,903 bytes/segundo, y un valor en el eje de ordenadas de 127.960.000 bytes / 234,560 segundos = 545.532,06 bytes/segundo

3.2- Generación de gráficas de latencia y throughput a partir de resultados de simulador.

En esta sección vamos a practicar lo explicado en la sección 3.1 a partir de ficheros de resultados. El estudiante deberá realizar 2 ejercicios de construccion de gráficas a partir de resultados de simulación en fichero de texto.

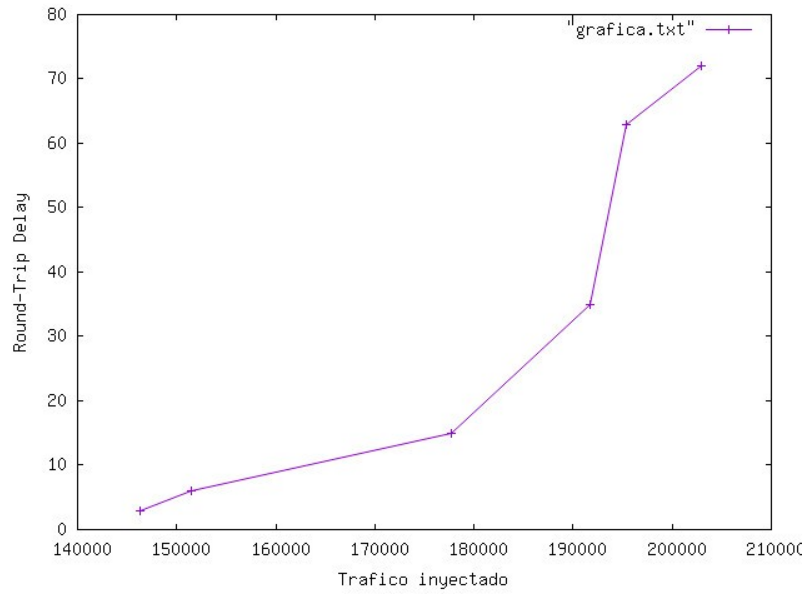
3.2.1 Generación de gráficas a partir de ficheros de resultados de proyecto.

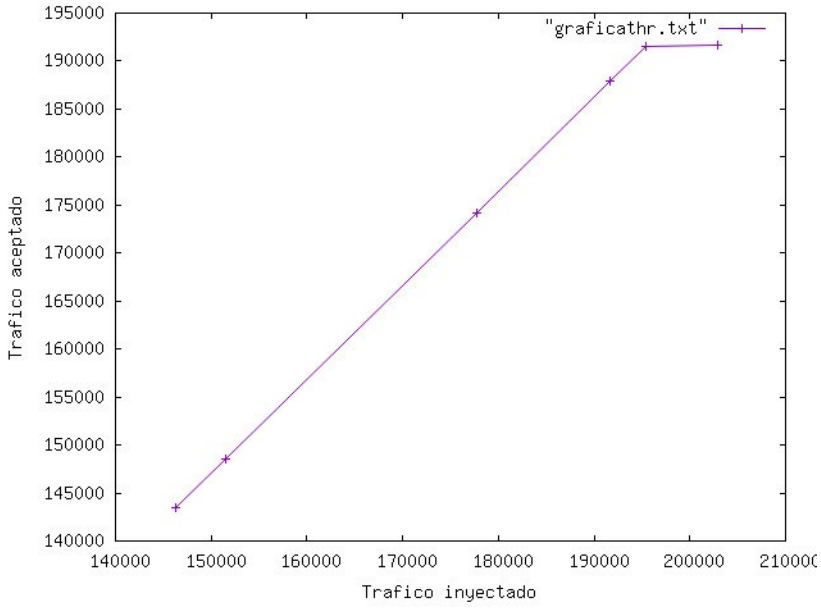
En este ejercicio el estudiante deberá bajarse los ficheros de resultados que proporciona un proyecto desarrollado por alumnos en cursos anteriores, y a partir de estos ficheros deberá construir las **gráficas de tiempo de respuesta y de throughput**. Para ello, puede utilizar cualquier herramienta: Libreoffice, Excel, Matlab, R, Gnuplot, etc..

Para generar la gráficas asumiremos que TODOS los mensajes enviados por los clientes para comunicar nuevas coordenadas tienen **10 bytes**, y TODOS los mensajes de reconocimiento tienen **3 bytes** de longitud.

En concreto el estudiante deberá usar los 6 ficheros de resultados que hay en el directorio de la práctica 1 denominado “resultados_proyecto”. Estos ficheros se denominan “xx_clientes_grupo100.txt”, donde “xx” va desde 100 hasta 2000, e indica el número de clientes que se han usado en cada simulación. Todos los ficheros (simulaciones) se han realizado con grupos de 100 vecinos y con 5 iteraciones, tal como figura en cada uno de los ficheros de resultados.

El estudiante deberá explicar en un informe cómo y con qué herramientas ha obtenido tanto la gráfica de tiempo de respuesta como la de throughput, así como analizar la gráfica (con cuántos clientes se satura el sistema, etc.). A efectos informativos, las gráficas obtenidas deberían ser similares a las que aparecen a continuación, que están sacadas de estos ficheros de resultados:



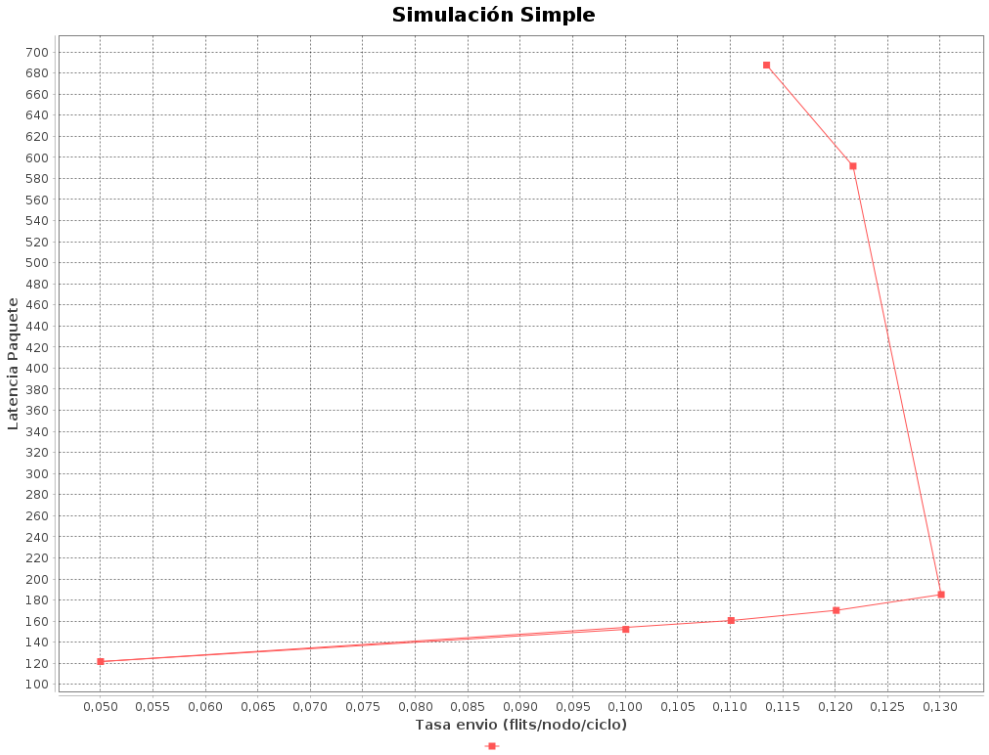


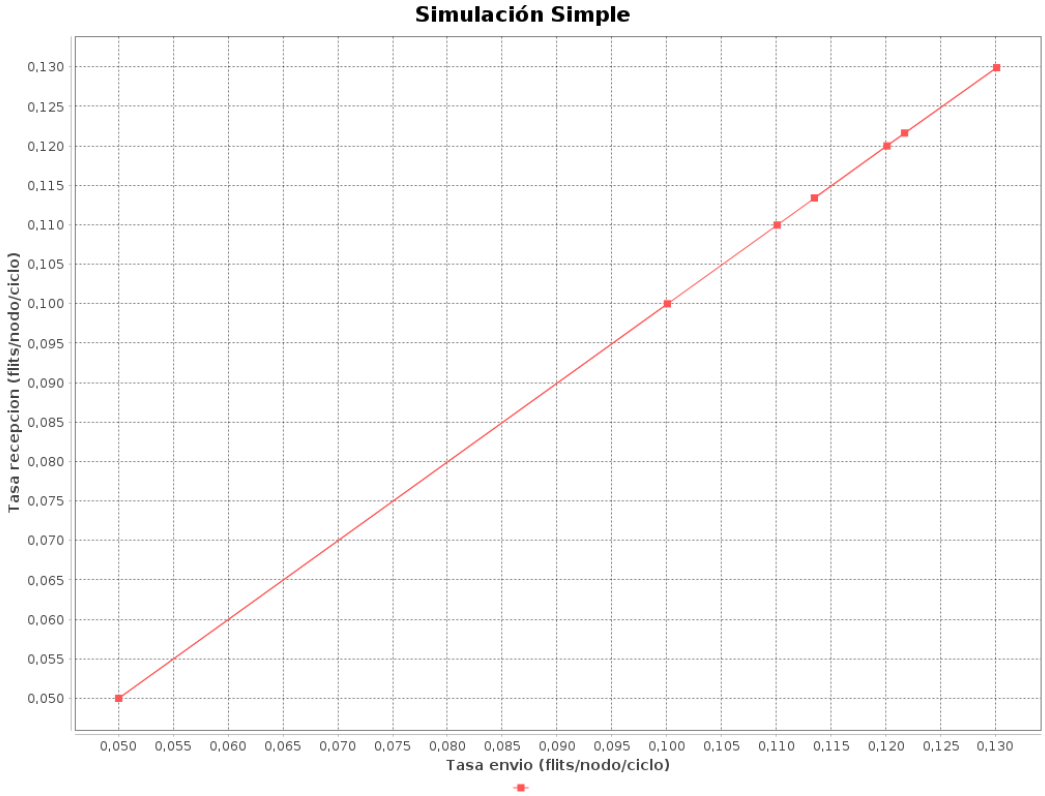
3.1.2 Generación de gráficas a partir de resultados del simulador Jsimured.

En este ejercicio el estudiante deberá bajarse los ficheros de resultados que proporciona el simulador Jsimured, e igualmente a partir de estos ficheros deberá construir las **gráficas de tiempo de respuesta y de throughput**. Para ello, puede utilizar cualquier herramienta: Libreoffice, Excel, Matlab, R, Gnuplot, etc., e incluso el propio simulador Jsimured.

En concreto el estudiante deberá construir 2 gráficas, una de latencia media de paquetes y otra de throughput, a partir de cada uno de los 2 ficheros que hay en el directorio de la práctica 1 denominado "resultados del simulador Jsimured". Estos ficheros csv se denominan como el patrón de tráfico con el cual han sido generados.

El estudiante deberá explicar en un informe cómo y con qué herramientas ha obtenido tanto la gráfica de tiempo de respuesta como la de throughput, así como analizar la gráfica (con cuántos clientes se satura el sistema, qué patrón de tráfico ofrece menor latencia y/o mayor throughput, etc.). A efectos informativos, las gráficas obtenidas deberían ser similares a las que aparecen a continuación, que están sacadas de estos ficheros de resultados:





4- EVALUACIÓN DE LA PRÁCTICA

Al final de la sesión cada grupo de trabajo deberá subir a la tarea correspondiente en Aula Virtual un informe con las respuestas a las 10 preguntas que hay en el capítulo 2 de este guión, así como la elaboración y análisis de las gráficas del capítulo 3. La evaluación de la práctica consistirá en evaluar el informe subido a Aula Virtual. Cada pregunta del capítulo 2 valdrá **0,5 puntos**, para un total de 5 puntos. El informe sobre cada subapartado del capítulo 3 valdrá **2,5 puntos**, para un total de otros 5 puntos.

Las respuestas deberán contener las capturas de pantalla necesarias que corroboren o ilustren cada una de las respuestas.

En el informe subido a Aula Virtual deberá figurar el nombre de los componentes del grupo de trabajo que han realizado esta práctica. Si el nombre de alguno de los componentes del grupo no figura en dicho documento, la nota asignada a dicho estudiante será de 0 puntos sobre 10.